

AI Course

Team Project Action Plan

For students (instructor review required)

©2019 SAMSUNG. All rights reserved.

Samsung Electronics Corporate Citizenship Office holds the copyright of this document.

This document is a literary property protected by copyright law so reprint and reproduction without permission are prohibited.

To use this document other than the curriculum of Samsung innovation Campus, you must receive written consent from copyright holder.

Clasificación de imágenes para la detección de cáncer de mama

17/12/2021



[Listado de miembros]

Contenidos

1. Introducción
 - 1.1. Contexto del proyecto
 - 1.2. Motivación y objetivos
 - 1.3. Asignación de miembros y roles
 - 1.4. Calendario e hitos
2. Ejecución del proyecto
 - 2.1. Adquisición de datos (Sensor, Actuator, Controller)
 - 2.2. Metodología de trabajo
 - 2.3. Flujo de trabajo
 - 2.4. Diagrama del sistema
3. Resultados
 - 3.1. Procesamiento de datos
 - 3.2. Análisis de datos exploratorios
 - 3.3. Modelado
 - 3.4. Interfaz de usuario
 - 3.5. Pruebas y mejoras
4. Proyección del impacto
 - 4.1. Logros y beneficios
 - 4.2. Futuras mejoras
5. Revisión y comentarios de los miembros del equipo
6. Revisión de los tutores y comentarios

1.1. Contexto del proyecto

El cáncer de mama es una proliferación celular maligna que se origina en la glándula mamaria. En España el cáncer de mama es el tipo de cáncer más frecuentemente diagnosticado en las mujeres, se estima que en el año 2021 se diagnosticaron 33.375 casos nuevos de cáncer de mama. En el año 2020 fallecieron 6.606 mujeres por cáncer de mama.

1.2 Motivación y objetivos

- Motivación:

Nuestra mayor motivación para la realización del proyecto, fué hacer algo relacionado con las mujeres, ya que el curso que estamos realizando es exclusivamente para ellas. Nos parecía que hacer algo relacionado con la sanidad, sería algo muy funcional, y la IA podría ser un gran apoyo de cara a encontrar un diagnóstico precoz. Estudios demuestran que cuanto antes se inicia el tratamiento, la probabilidad de ganar a la enfermedad es más alta.

- Objetivos:

En este trabajo, se pretende aplicar las técnicas de deep learning para diseñar y desarrollar algoritmos que sean capaces de detectar a partir una imagen histopatológica, si hay presencia de carcinoma ductal invasivo (CDI) en dicha imagen, que es el tipo de cáncer de mama más común, estando presente en el 80% de los casos de cáncer de mama detectados.

El principal objetivo es diseñar e implementar un modelo basado en redes neuronales que permita clasificar imágenes histopatológicas con un accuracy de al menos un 75%. Como objetivo secundario, sería poder proporcionar una herramienta que complemente el procedimiento habitual médico, para el diagnóstico del carcinoma, realizando un diagnóstico rápido y fiable mediante la clasificación de las imágenes en positivo en CDI o negativo en CDI.

1.3 Asignación de miembros y roles

AAAAA: Procesamiento de datos y modelos. Jefa de proyecto.

BBBBB : Procesamiento de datos y. Miembro de equipo.

CCCCC: Procesamiento de datos y modelos. Miembro de equipo.

DDDDD: Interfaz. Miembro de equipo.

1.4 Calendario e hitos

Iniciación: 16/11/2021

Planificación: 17/11/2021 - 25/11/2021

Desarrollo del proyecto: 25/11/2021 - 3/12/2021

Ejecución: 4/12/2021 - 17/12/2021

Control del proyecto: 4/12/2021 - 17/12/2021

Capstone Project Work Breakdown Structure (WBS)																									
Today		16/12/2021																							
Project Team Name		Mama Scan																							
Project Topic		Clasificación de cáncer de mama																							
Task	Start date	End date	Responsibility	Deliverable	Task Status	W0				W1				W2				W3				W4			
						D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	D11	D12	D13	D14	D15	D16	D17	D18	D19	D20
1. Iniciación	16/11/2021	16/11/2021	Cristina		Done	-																			
1.1. Creación grupo de Telegram	16/11/2021	16/11/2021	Cristina		Done	-																			
1.2. Creación grupo Discord	17/11/2021	17/11/2021	Cristina		Done	-																			
2. Planificación	17/11/2021	25/11/2021	Equipo		Done	-																			
2.1. Determinación del tema del proyecto	17/11/2021	22/11/2021	Equipo		Done	-																			
2.1.1. Brainstorming	17/11/2021	20/11/2021	Equipo		Done	-																			
2.1.2. Validación de la idea	17/11/2021	20/11/2021	Equipo		Done	-																			
2.1.3. Confirmación del proyecto	20/11/2021	21/11/2021	Equipo		Done	-																			
2.1.4. Selección de metodología de gestión del proyecto	22/11/2021	22/11/2021	Equipo		Done	-																			
2.2. Búsqueda de información	17/11/2021	25/11/2021	Equipo		Done	-																			
2.2.1. Dataset en Kaggle	17/11/2021	25/11/2021	Equipo		Done	-																			
2.2.2. Proyectos relacionados	17/11/2021	25/11/2021	Equipo		Done	-																			
2.3. Realización de informes	17/11/2021	25/11/2021	Equipo		Done	-																			
2.3.1. Diagrama de Gantt	17/11/2021	25/11/2021	Equipo	si	Done	-																			
2.3.2. Elaboración del Action Plan	17/11/2021	25/11/2021	Equipo	si	Done	-																			
2.3.3. Presentación de objetivos	17/11/2021	25/11/2021	Equipo	si	Done	-																			
3. Desarrollo del proyecto	25/11/2021	3/12/2021	Equipo		Done	-																			
3.1. Datos	25/11/2021	26/11/2021	Equipo		Done	-																			
3.1.1. Cargar datos	25/11/2021	26/11/2021	Equipo		Done	-																			
3.1.2. Limpieza de datos	25/11/2021	26/11/2021	Equipo		Done	-																			
3.1.3. División de datos	25/11/2021	26/11/2021	Equipo		Done	-																			
3.2. Modelo	26/11/2021	2/12/2021	Equipo		Done	-																			
3.2.1. Tutorial de clasificación de imágenes	26/11/2021	2/12/2021	Equipo		Done	-																			
3.2.2. Creación de un primer modelo	26/11/2021	2/12/2021	Equipo		Done	-																			
3.3. Training and Testing	26/11/2021	15/12/2021	Equipo		Done	-																			
3.3.1. Entrenamiento	26/11/2021	15/12/2021	Equipo		Done	-																			
3.3.2. Testing (Accuracy)	2/12/2021	15/12/2021	Equipo		Done	-																			
4. Ejecución	4/12/2021	17/12/2021	Equipo		Done	-																			
4.1. Visualización del proyecto	4/12/2021	17/12/2021	Equipo		Done	-																			
5. Control del proyecto	4/12/2021	17/12/2021	Equipo		Done	-																			
5.1. Memoria del proyecto	22/11/2021	17/12/2021	Equipo	si	Done	-																			
5.2. Presentación final	17/12/2021	17/12/2021	Equipo	si	Done	-																			

2. Ejecución del proyecto

2.1 Adquisición de datos

Obtuvimos nuestro dataset de Kaggle:

<https://www.kaggle.com/paultimothymooney/breast-histopathology-images/code>

2.2 Metodología de trabajo

Se ha intentado establecer una manera de trabajar muy colaborativa, fomentando el trabajo en equipo. Por ello se ha pensado en una metodología ágil, tipo scrum para implementarla en este proyecto.

Al inicio del proyecto se han recogido los requisitos que había que cumplir y plazos. Al comienzo de una iteración nueva (coincidiendo en fecha con la finalización de las presentaciones) el equipo ha seleccionado uno o varios requisitos en función de cuántos podrían ser implementados en una iteración.

El equipo se ha reunido diariamente realizando reuniones de planificación al comienzo de la sesión donde se establecen las tareas específicas a realizar y se reparten las tareas.

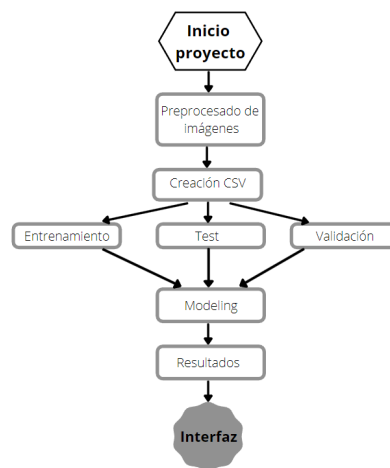
Durante las sesiones el equipo se autoorganiza para cumplir con el objetivo.

Se han realizado 3 reuniones de retrospectiva, 2 después de cada presentación, donde se han discutido aspectos que había que mejorar del último ciclo que se acababa de completar.

2.3 Flujo de trabajo

- Creamos nuestro diagrama de Gantt para ordenar y programar nuestro día a día en nuestro proyecto.
- Reunirnos todos los días para planificar nuestro trabajo.
- Solucionar todos los problemas que nos van surgiendo.
- Debatir ciertas mejoras del modelo.

2.4 Diagrama del Sistema



3. Resultados

3.1. Procesamiento de datos

Es importante tener un conjunto de datos bien preparado. En esta fase se han procesado los datos para que se adecuen a las necesidades de los modelos que van a ser entrenados, haciendo que estos sean eficaces y den resultados correctos.

Se han realizado 3 pasos para preparar los datos:

1.- División del conjunto de datos (Conjunto de entrenamiento, de validación y de test)

Aunque la mayoría de las veces la división del conjunto de datos (entrenamiento, validación y test) se realiza de forma aleatoria, en ciertas ocasiones nos interesa realizar una división lineal, como es en nuestro caso para mantener las imágenes de un mismo paciente en un mismo conjunto de datos y así evitar datos falsos de Accuracy por la correlación entre dichas imágenes.

Datos de entrenamiento que se utilizarán para entrenar las redes neuronales.

Datos de validación que se utilizarán para validar y optimizar los resultados de las redes neuronales durante la fase de entrenamiento, sintonizando y reajustando los hiperparámetros.

Datos de test que se utilizarán después de entrenar las redes neuronales para asegurarse de que el modelo esté generalizado y no sobreajustado. Nunca se usa durante el entrenamiento.

2.- Identificación de sesgos. Datos desbalanceados.

Un sesgo es un peso desproporcionado en el que una muestra se recopila de tal manera, que algunos miembros de la población prevista tienen menos probabilidades de ser incluidos que otros. En el caso de muestras sobre el área de salud, suele ser bastante habitual que ocurra esto, es decir, se puede contar con muchos registros de paciente “negativos” y pocos registros de pacientes “positivos” (padecen la enfermedad que se quiere clasificar).

En nuestro problema de clasificación, donde etiquetamos entre “Benigno” (valor 0) o “Maligno” (valor 1) nos encontramos con que, en el conjunto de entrenamiento, existe una clase claramente minoritaria que es la clase “Maligno”.

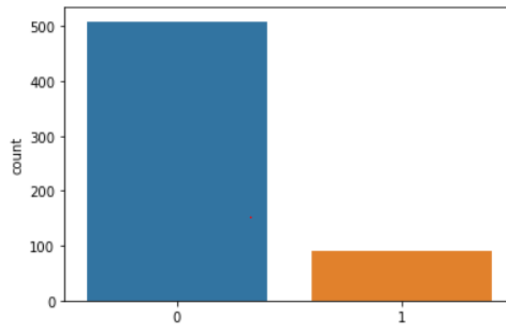


Figura1. Histograma conjunto de datos de entrenamiento según clases (0 = benigno, 1 = maligno).

Esta circunstancia afecta a los modelos en el momento de generalizar la información, no logrando diferenciar claramente una clase de otra y perjudicando por tanto a la clase minoritaria. Hemos probado diferentes estrategias para el manejo de datos desbalanceados y elegido el que mejor resultado obtuvo: *(modelo seleccionado para la prueba MobilNetV2. Matriz de confusión y métricas (Accuracy, precisión, recall) para analizar los resultados).*

a) No balanceo de datos.

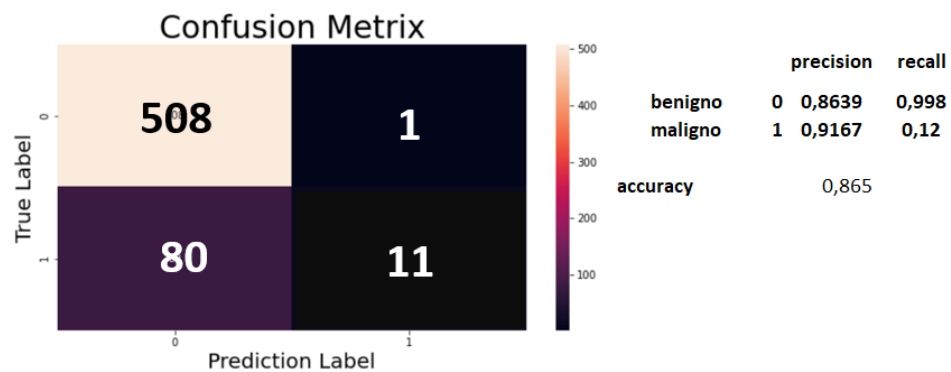


Figura2. Matriz de confusión y métricas sobre la estrategia de no balanceo

Se observa que el modelo no es capaz de detectar los casos malignos, ya que para la clase 1, ha acertado 11 frente a 80 que ha fallado. Aunque el accuracy es bueno, el recall para la clase 1 que es de solo 0.12, indica que el modelo no logra detectar esa clase.

b) Subsampling en la clase mayoritaria:

Se reduce la clase mayoritaria (clase 0) para balancear los datos. Hemos utilizado el algoritmo NearMiss para seleccionar los registros que va a eliminar.

El resultado:

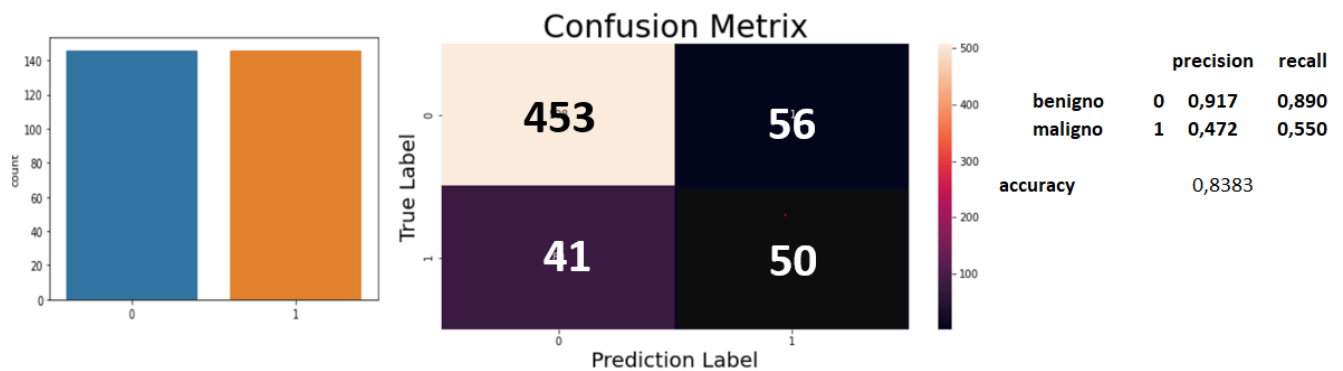


Figura3. Histograma de clases, matriz de confusión y métricas sobre la estrategia de subsampling.

Se observa que hay una mejoría importante. Ahora el modelo detecta mucho mejor los casos malignos, ya que para la clase 1, ha acertado 50 frente a 41 que ha fallado, con un recall de 0.55. Aunque hay un aumento de los falsos positivos.

c) Oversampling en la clase minoritaria:

Se aumenta la clase minoritaria (clase 1) con datos “creados” a través del algoritmo RandomOverSample. El resultado:

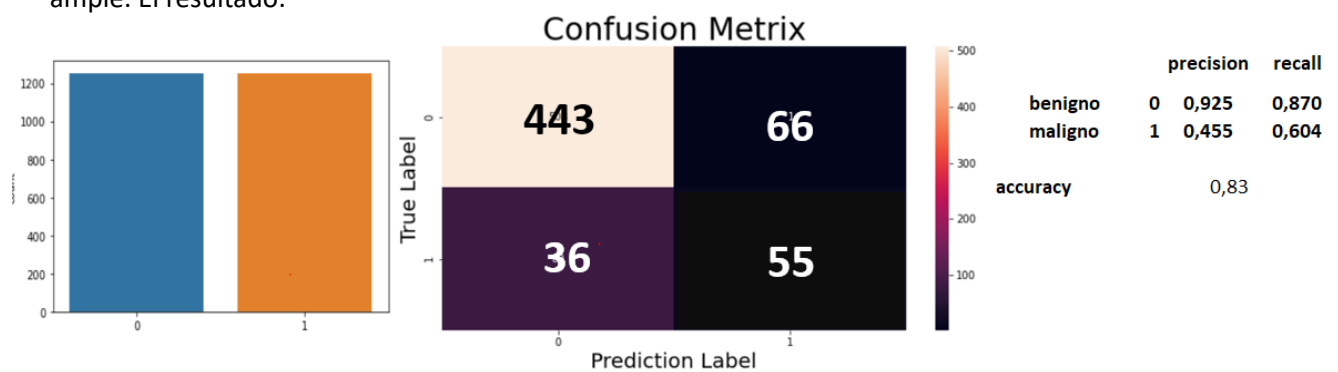


Figura4. Histograma de clases, matriz de confusión y métricas sobre la estrategia de oversampling.

Ahora tenemos un recall mejorado en la clase 1 de 0.604.

d) Resampling con Smote-Tomek:

Se aplica a la vez un algoritmo de subsampling y otro de oversampling mediante el algoritmo SMO TE para oversampling y Tomek para el subsampling.

El resultado:

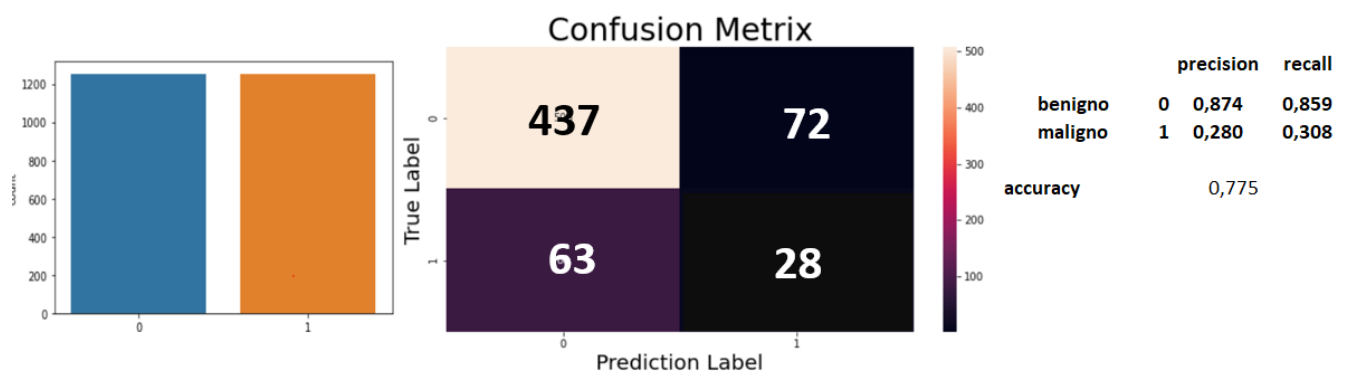


Figura5. Histograma de clases, matriz de confusión y métricas sobre la estrategia de resampling.

En este caso, el recall en la clase 1 ha empeorado considerablemente con respecto al anterior, por lo que no sería una buena estrategia para nuestra clasificación.

Resultado de las estrategias:

	precision	recall
No balanceo	0,92	0,12
Subsampling	0,47	0,55
Oversampling	0,46	0,60
Resampling	0,28	0,31

El mejor resultado obtenido es con la estrategia del oversampling y es este el que tomaremos en el proyecto.

3.- Transformación de los datos.

En este punto, lo que se ha realizado es una transformación de los datos del dataset, que en nuestro caso son imágenes a datos entendibles en el marco del Machine learning y del Deep learning.

- Convertir las imágenes en matrices de píxeles.***
- Cambiar el tamaño de las imágenes.*** El tamaño de las imágenes del dataset es de 50x50. Varios modelos de los que vamos a probar posteriormente funcionan de manera más eficiente con imágenes de tamaño 224 x 224. En este punto decidimos adecuar el tamaño de las imágenes a este y mantener las imágenes a color, es decir con 3 canales. Por lo tanto, cambiamos las imágenes de (50 X 50) a (224 X 224 X 3). Esta transformación suele consumir muchos recursos de memoria.
- Normalización de los datos.*** Hacemos que las dimensiones de los datos tengan la misma escala. Esto hace que sea más fácil entrenar las redes neuronales, especialmente las redes neuronales convolucionales.
- One-hot-encoding para la variable de clases.*** En nuestro caso únicamente tenemos 2 clases por lo que este punto no sería completamente necesario. Pero puede ayudar a ofrecer un conjunto de predicciones más matizado que una sola etiqueta.

Con la codificación one-hot lo que se hace es convertir la variable categórica, “diagnóstico”, como vectores binarios.

3.2 Análisis de datos exploratorios. EDA

Analizamos los datos para resumir sus principales características.

Recordar en este punto que los tumores ocupan solo una pequeña parte de la imagen completa histopatológica de la biopsia extraída. Una imagen digital de campo completo suele tener 4000 x 3000 píxeles, mientras que una región de interés potencialmente cancerosa (ROI) puede ser tan pequeña como 100 x 100 píxeles.

En primer lugar, visualizamos algunas imágenes del dataset:

En esta primera figura, tenemos una selección de imágenes de diagnóstico negativo en ICD.

Imágenes diagnosis Negativo

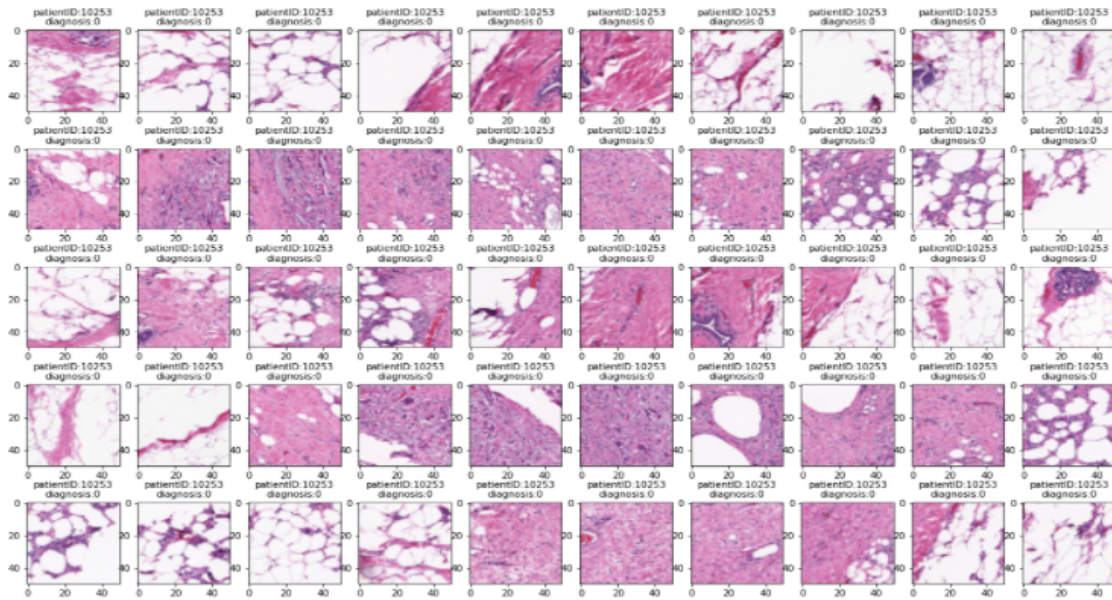


Figura6. Colección de muestras de imágenes de diagnóstico benigno.

En esta segunda figura, tenemos una selección de imágenes de diagnóstico positivo en ICD.

Imágenes diagnosis Positivo

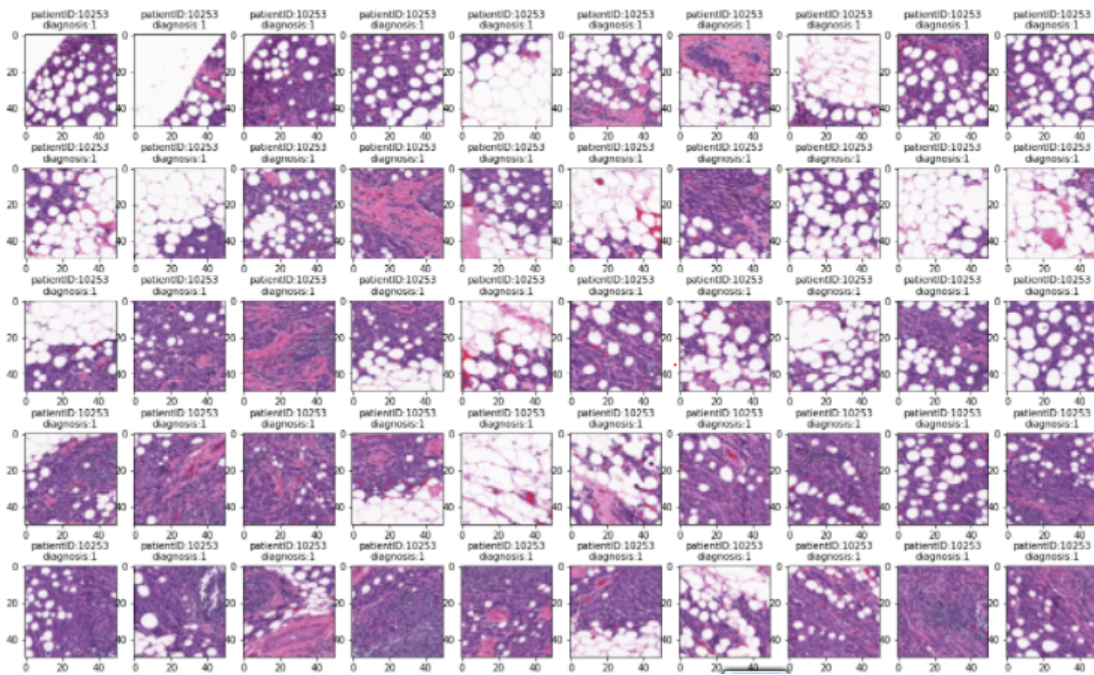


Figura7. Colección de muestras de imágenes de diagnóstico maligno.

A simple vista y sin ser un experto es complicado diferenciar entre una imagen de diagnóstico benigno y maligno, ya que no se observan características muy obvias en todas. Quizá lo único que puede resaltar es un tono de color más intenso en las malignas, junto con una zona más densa. Vemos ahora un ejemplo de composición completa de la biopsia realizada a una paciente, donde la zona gris es tejido sano y la zona resaltada en violeta tejido canceroso.

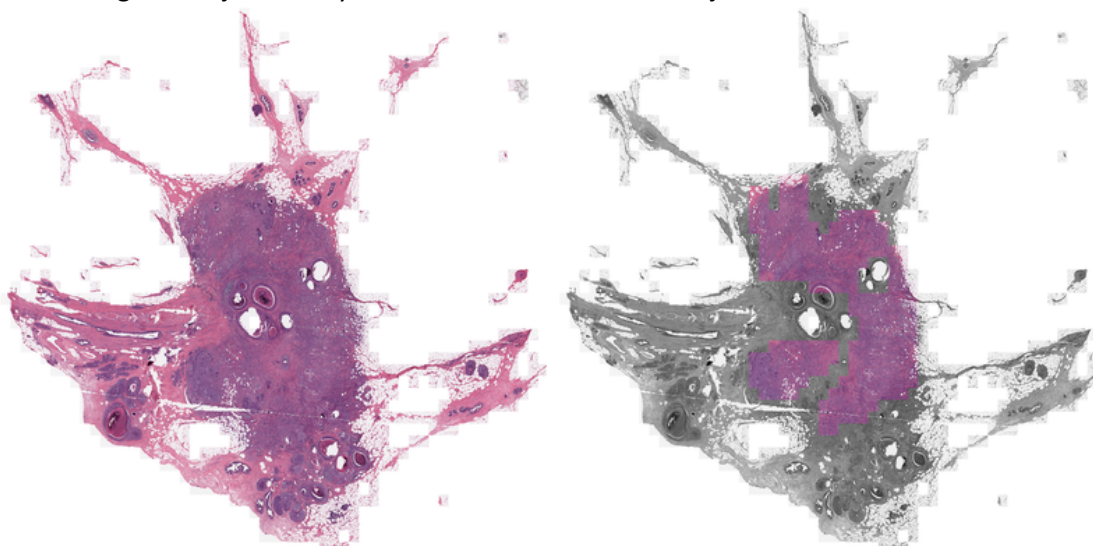
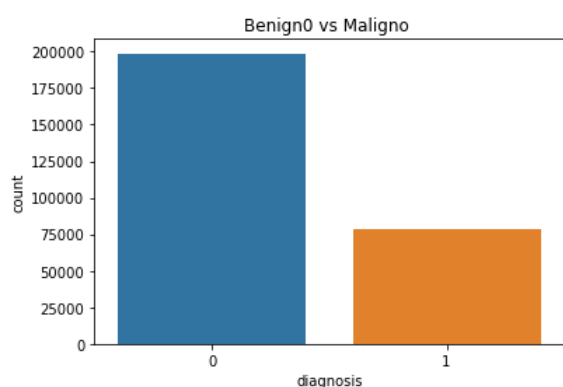


Figura8. Composición completa de la biopsia realizada a la paciente 8863.

Las imágenes tienen dimensiones no uniformes y, a menudo, contienen múltiples cortes de tejido de diferentes tamaños, rodeados por grandes regiones vacías. Además, no son muy homogéneas entre sí, existe una gran variedad.

Continuamos con el análisis de los datos:

- Datos descompensados por tipo de clase:



La distribución de los datos es de un 28% de imágenes malignas frente a un 72% de imágenes benignas.

0	198738
1	78786

La diferencia en cantidad de benigno frente a maligno es de: 119952

Figura9. Frecuencia de imágenes por clase.

- Características de los datos por paciente:

Al agrupar por paciente, se observa que el número de imágenes por paciente no es homogéneo. Va desde un mínimo de 63 imágenes hasta un máximo de 2395 imágenes.

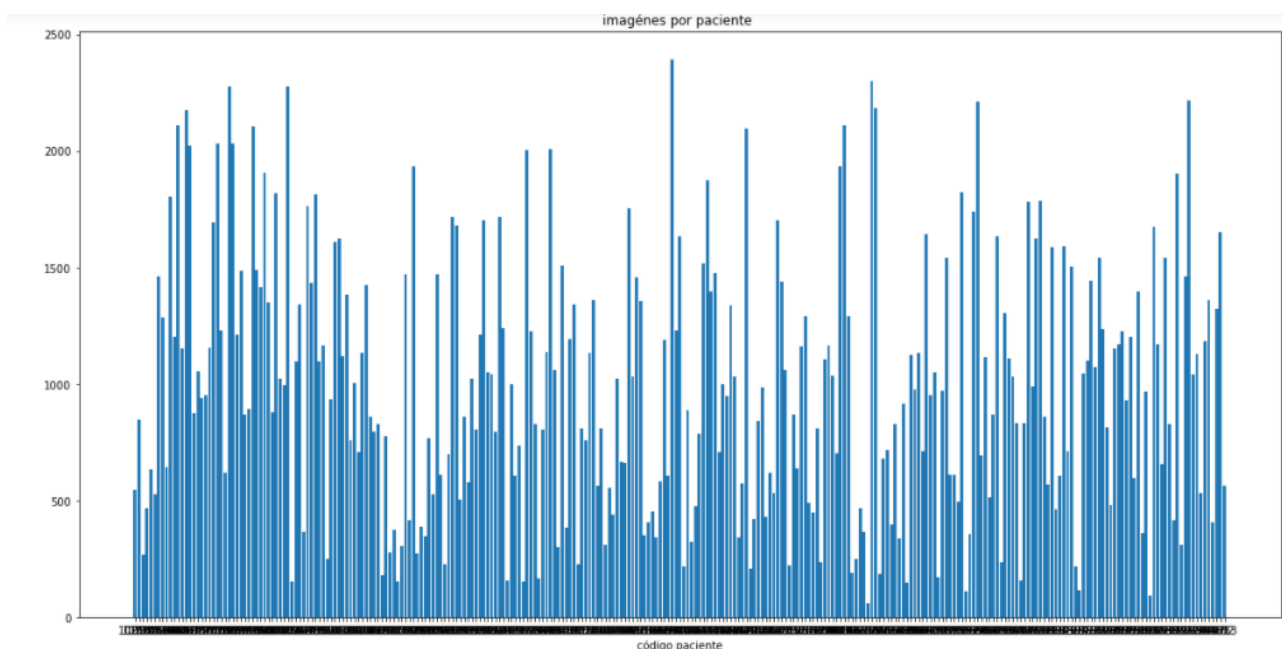


Figura10. Frecuencia de imágenes por paciente.

Lo mismo ocurre con el número de imágenes positivas y negativas por paciente. No son homogéneas. Va desde un mínimo de 10 hasta un máximo de 2231.

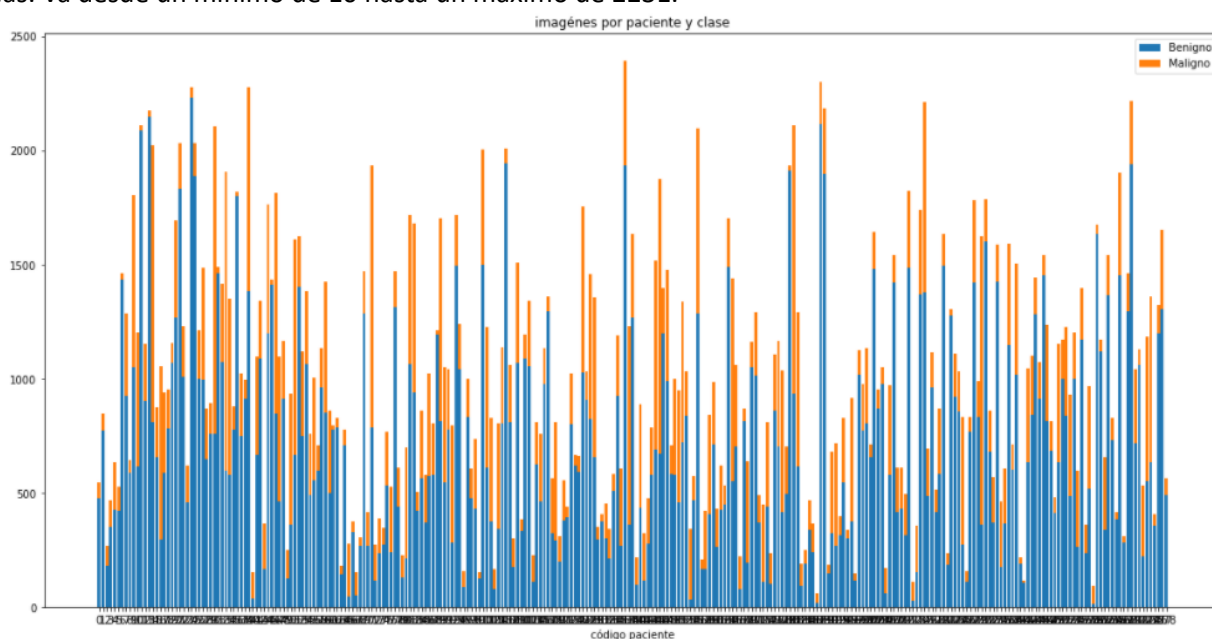


Figura11. Frecuencia de imágenes por paciente y diagnóstico.

3.3 Modelado

Después de dividir, normalizar y convertir el conjunto de datos, ahora vamos a construir el modelo .

CONSTRUCCIÓN DEL MODELO.

De la gran variedad de redes neuronales que existen, vamos a probar para nuestro proyecto de clasificación, las que han demostrado ser más eficaces en el procesamiento de imágenes médicas, que es la particularidad del proyecto.

Las redes neuronales convolucionales (CNN) se utilizan principalmente para el procesamiento de i

márgenes y la detección de objetos. Son utilizadas ampliamente para procesar imágenes médicas y han probado ser muy eficientes en tareas de clasificación y reconocimiento de imágenes.

Por lo tanto, se van a explorar las redes neuronales convolucionales (CNN). Para realizar dicha exploración, se han realizado pruebas sobre una red neuronal convolucional estándar y sobre arquitecturas de redes convolucionales más complejas empleando la técnica de transfer-learning.

En este trabajo, una vez implementada la red neuronal convolucional estándar (CNN), se ha procedido a realizar una exploración sobre ella alternando diversos parámetros como:

1. Número de épocas.
2. Batch_size, learning_rate o tamaño de filtros de las capas convolucionales.
3. Tamaño de las imágenes.

De esta manera se han comprobado los resultados obtenidos de las diversas ejecuciones, y se ha estimado cuál sería la configuración óptima obtenida.

Además, se ha utilizado la técnica de transfer-learning con arquitecturas complejas predefinidas para explorar y evaluar sus resultados con una combinación diferente de parámetros como:

1. El learning rate,
2. El tamaño del batch
3. O el número de capas adicionales.

Una vez realizadas las pruebas y la evaluación de los resultados, se realiza una comparación con los resultados obtenidos en la CNN mencionada en el párrafo anterior.

Veámoslo:

MODELO CNN

Las CNN tienen múltiples capas que procesan y extraen características de los datos. En este trabajo, la red CNN creada consta, en su primera parte de extracción de características, de una primera red convolucional con 32 filtros, para un tamaño de kernel de 5x5 que filtra la imagen de entrada multiplicándose de forma matricial. La salida después de esta primera convolución es consistente con el tamaño de entrada, ya que el padding = same.

A continuación, se añade una capa de agrupación (maxpooling) de tamaño 2, que reduce las dimensiones del mapa de características, conservando las características más importantes que detectó cada filtro. De nuevo otra capa de convolución similar a la primera, que recogerá características de formas más complejas. Para posteriormente aplicar otra capa de agrupamiento (maxpooling), de tamaño 2 que reducirá de nuevo la dimensión del mapa, obteniendo ya el mapa de detección de características. A esta, sigue otra capa de agrupación (flatten) que convierte la matriz tridimensional resultante del mapa de características en un vector lineal, aplanándolo.

Ahora pasaríamos ya a la fase de clasificación, con una capa densa (completamente conectada) que se alimenta como entrada de la matriz aplanada, seguida de un dropout para el 70%. Y por último otra capa densa a la que se aplica la función Softmax que conecta contra la capa de salida final, que es de 2 neuronas, ya que tenemos 2 clases para la salida (benigno y maligno).

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 224, 224, 32)	2432
max_pooling2d (MaxPooling2D)	(None, 112, 112, 32)	0
conv2d_1 (Conv2D)	(None, 112, 112, 64)	51264
max_pooling2d_1 (MaxPooling2D)	(None, 56, 56, 64)	0
flatten (Flatten)	(None, 200704)	0
dense (Dense)	(None, 1024)	205521920
dropout (Dropout)	(None, 1024)	0
dense_1 (Dense)	(None, 2)	2050
Total params: 205,577,666		
Trainable params: 205,577,666		
Non-trainable params: 0		

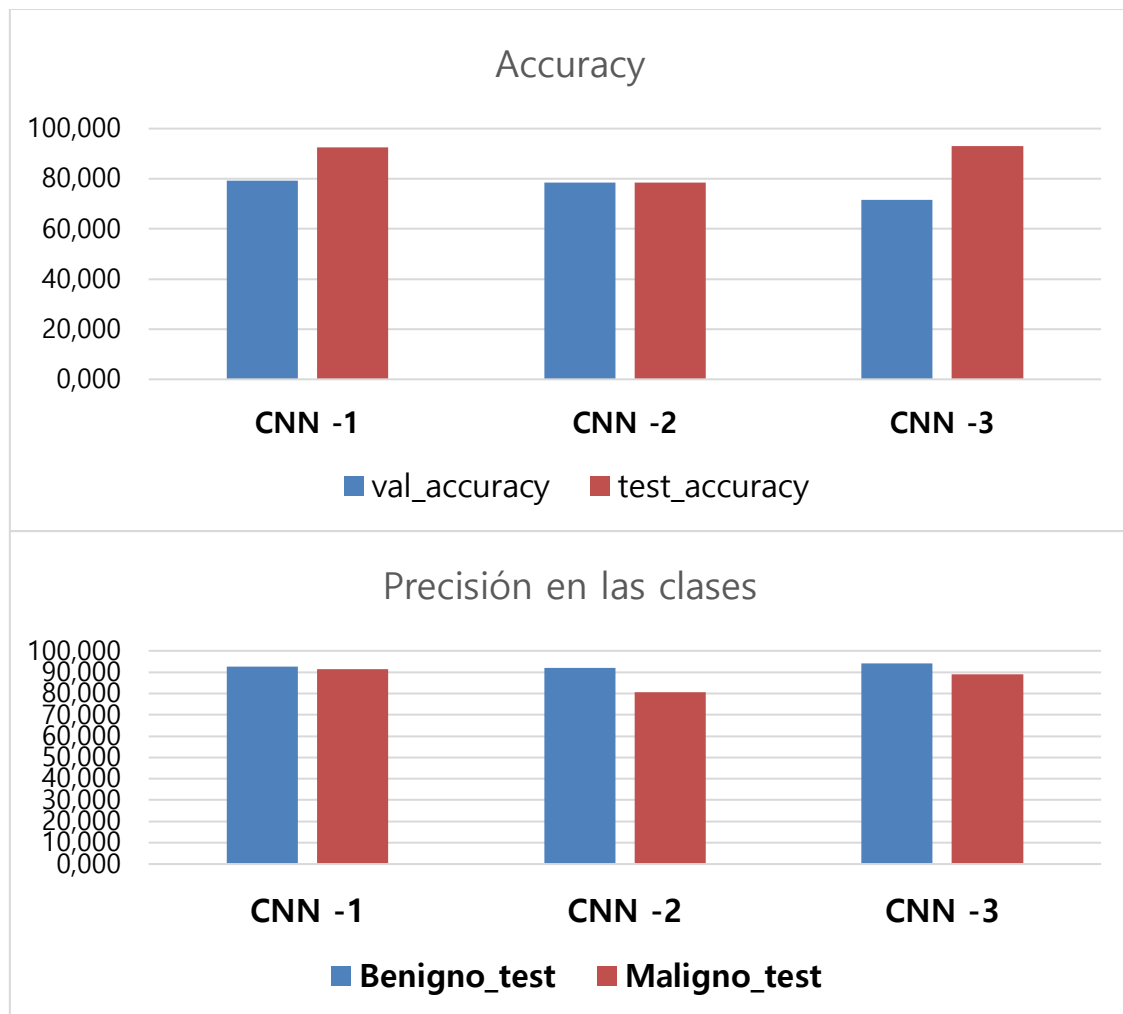
DIAGNÓSTICO DEL RENDIMIENTO. PRUEBAS Y RESULTADOS

Se van a utilizar las gráficas de las curvas de aprendizaje como herramienta de diagnóstico, así como las métricas accuracy, loss y precisión. El modelo se evaluará en el conjunto de datos de entrenamiento y en el conjunto de datos de validación. Y finalmente en un conjunto de datos de test se comprobará si el modelo está generalizando bien.

Se han realizado varias iteraciones con una combinación diferente de parámetros, obteniendo los siguientes resultados:



CNN - 1	CNN - 2	CNN - 3
epocas = 20	epocas = 20	epocas = 10
lr = 0,0002	lr = 0,0001	lr = 0,0001
batch_size = 32	batch_size = 40	batch_size = 64
optimizer = Adam	optimizer = Adam	optimizer = Adam
Dropout = 0,5	Dropout = 0,7	Dropout = 0,7



El comportamiento de estas gráficas es característico de un modelo que presenta overfitting. Las dos líneas que representan los datos de entrenamiento y validación en las gráficas de accuracy tienen una brecha considerable entre ellas que en algunos casos incluso se van aumentando con las épocas (CNN-2 y CNN-3). En todas, el Accuracy de los datos de entrenamiento aumenta linealmente con las épocas, hasta alcanzar prácticamente el 100%, mientras que el Accuracy de los datos de validación se estanca o incluso disminuye progresivamente. El Accuracy del entrenamiento es muy alto, pero el de validación no tanto.

A pesar del overfitting que presupone que el modelo no será capaz de devolver una buena salida sobre un conjunto nuevo, los resultados en test no son para nada malos. Al evaluar estos 3 modelos con los datos de test, se obtienen unos accuracy superiores al 92%. Por lo tanto, el modelo parece estar generalizando bien, aun cuando no lo ha hecho en la validación.

Esto puede ser debido a las particularidades del conjunto de validación seleccionado, ya que son únicamente de un paciente y en no mucha cantidad.

Para mitigar este sobreajuste, además de la modificación de hiperparámetros que se ha realizado, se va a implementar la validación cruzada, cuya idea es reducir la probabilidad de que el modelo final esté sobreajustado. En una validación cruzada se toman muchos subconjuntos del conjunto de entrenamiento y se entrena el modelo con dichos subconjuntos. De esta manera las particularidades de nuestro conjunto de validación se diluyen.

Hay que comprobar este aspecto, por lo que se procede a realizar un K-Fold cross validation, sobre el modelo CNN-1(modelo con mejores resultados). En concreto un GroupKFold, que es una variación de k-fold que asegura que el mismo grupo k, no esté representado tanto en los conjuntos de v

validación como en los de entrenamiento. GroupKFold permite detectar situaciones de overfitting como cuando el modelo aprende de características muy específicas de la paciente y podría fallar en generalizar en nuevas pacientes.

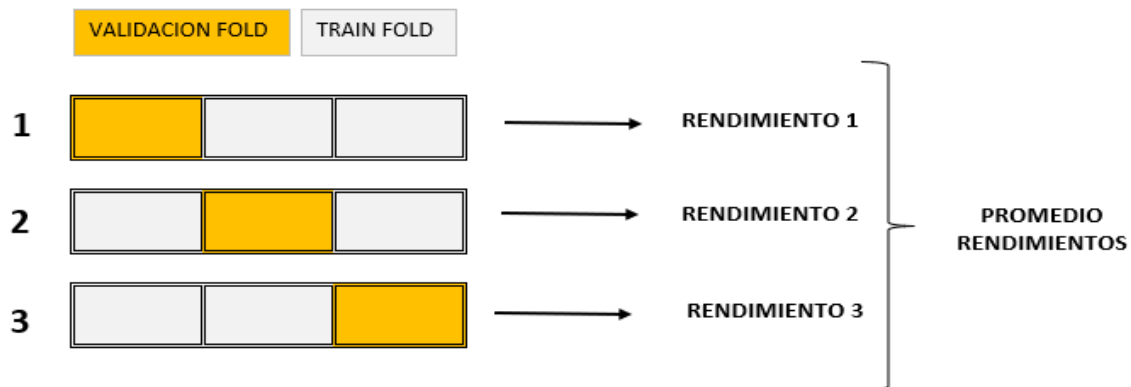


Figura. Esquema funcionamiento de la metodología K-Fold para K=3.

El método de evaluación ha constado de un proceso de $k=3$. El tamaño de k no puede ser superior a 3, ya que en el conjunto de entrenamiento únicamente hay datos de 3 pacientes y estos no se pueden mezclar al realizar la división entre train-validation.

El resultado obtenido ha sido bueno, consiguiendo un val_accuracy promedio de 81.68% durante el entrenamiento. El haber obtenido un val_accuracy de 84,5% en la primera iteración, 76% en la segunda y 84,6% en la última, muestra que la brecha train-validation es bastante menor en primera y última división, siendo la central el caso de nuestro conjunto de validación. Al evaluar el modelo sobre el conjunto de test, se ha obtenido un accuracy de 92.52% con una precisión en la clase benigno de 0.935 y en maligno de 0.93

Esta metodología confirma que el modelo CNN-1 entrenado bajo metodología k-fold es un buen modelo, que consigue generalizar bien sobre conjuntos de datos nuevos.

MODELOS PRE-ENTRENADOS (TRANSFER-LEARNING)

Es habitual partir de una red previamente entrenada y ajustar los valores de los parámetros para el caso específico de clasificación que se desea abordar.

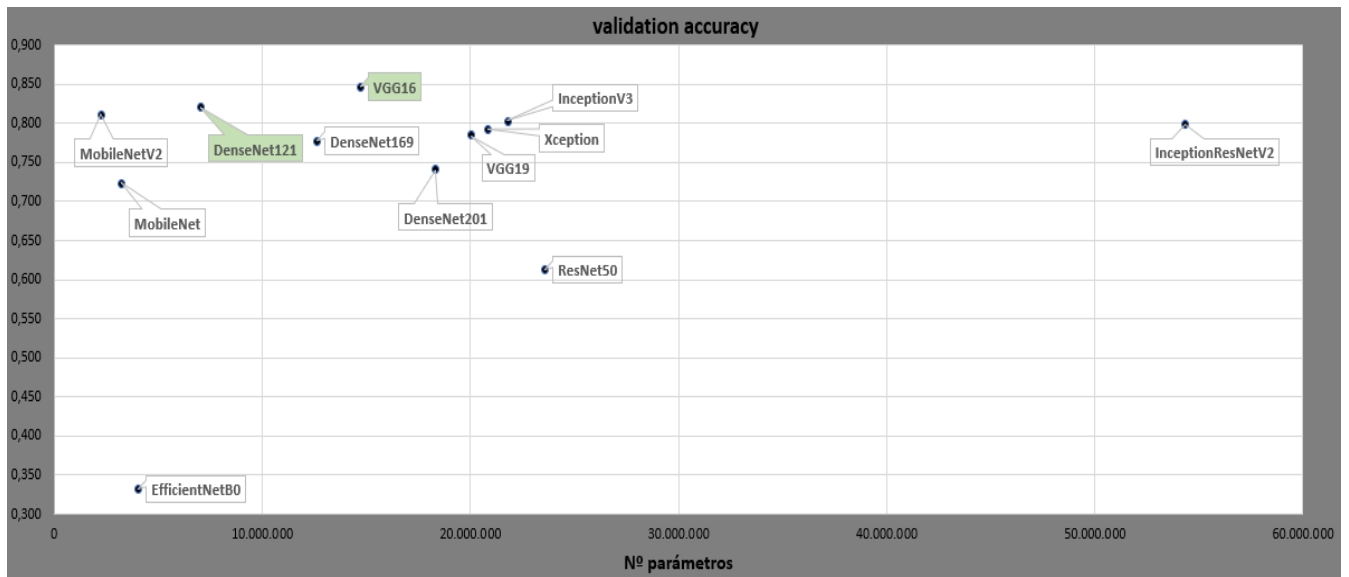
En este apartado vamos a hacer uso de la metodología Transfer Learning. Esta se basa en utilizar modelos entrenados previamente para crear el nuevo modelo. Transfer Learning tiene dos ventajas principales, un menor tiempo de entrenamiento y un menor número de datos requeridos para el entrenamiento.

Con intención de obtener el mejor rendimiento posible, en este trabajo vamos a hacer una primera aproximación a 12 de ellos. Actualmente en la base de keras hay 28 modelos pre-entrenados disponibles.

Todos los modelos que vamos a explorar en este apartado están pre-entrenados con el conjunto de datos ImageNet. Estos 12 modelos son:

* MobileNetV2	*MobileNet	*EfficientNetB0	*DenseNet121
*DenseNet169	*DenseNet201	*InceptionV3	*ResNet50
*InceptionResNetV2	*VGG16	*VGG19	*Xception

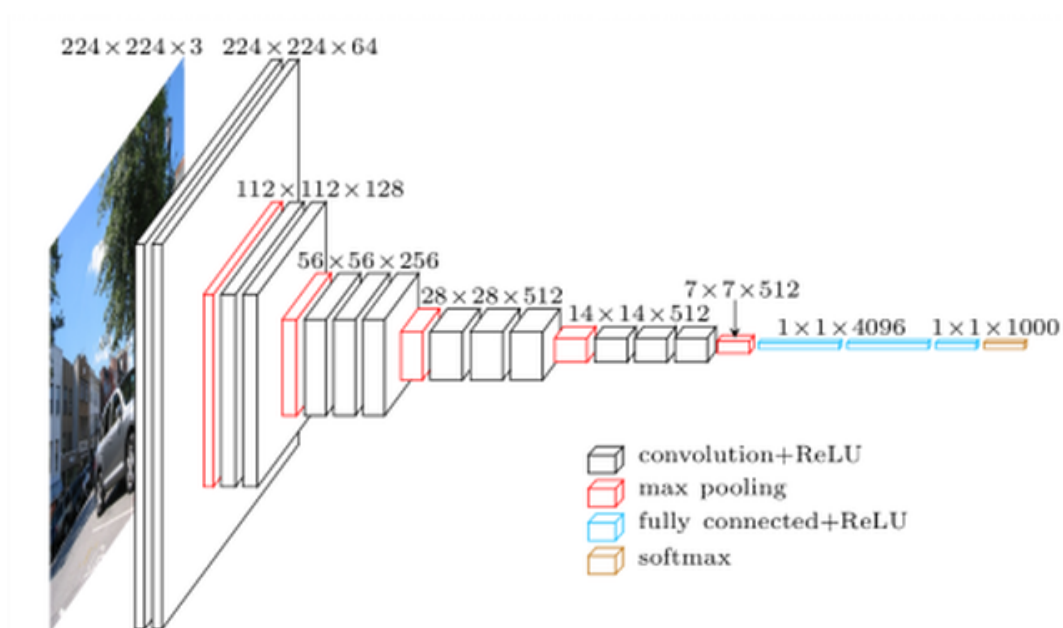
Entrenados en las mismas condiciones el resultado ha sido:



Los mejores resultados obtenidos son para los modelos VGG16. En este trabajo vamos a explorar el modelo VGG16.

VGG16

La red pre-entrenada VGG16, contiene 13 capas convolucionales y 3 densas. En concreto, "tiene 5 grupos de convoluciones, cada uno con un núcleo de convolución 3X3. Después de cada grupo de convoluciones, se realiza una agrupación máxima de 2X2, seguida de tres capas completamente conectadas. Al entrenar una red de alto nivel, primero puede entrenar la red de bajo nivel e inicializar la red de alto nivel con el peso obtenido por la primera, lo que puede acelerar la convergencia de la red".



Para este trabajo se ha creado un modelo de transfer learning con el modelo pre-entrenado VGG16 para la etapa de extracción de características y para la etapa de clasificación, una capa densa de salida de 2 unidades con función softmax.

El siguiente esquema corresponde al modelo creado cuyo rendimiento evaluamos a continuación:

Model: "vgg16"

Layer (type)	Output Shape	Param #
input_1 (InputLayer)	[(None, 224, 224, 3)]	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590880
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590880
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1180160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2359808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2359808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2359808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2359808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)	0

Total params: 14,714,688

Trainable params: 0

Non-trainable params: 14,714,688

Model: "sequential_1"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 512)	14714688
dense (Dense)	(None, 2)	1026

Total params: 14,715,714

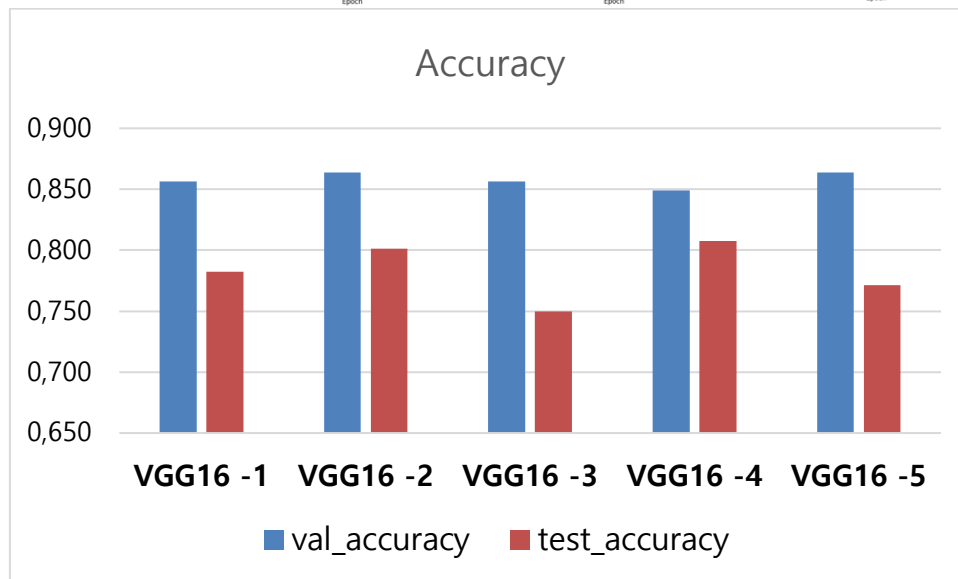
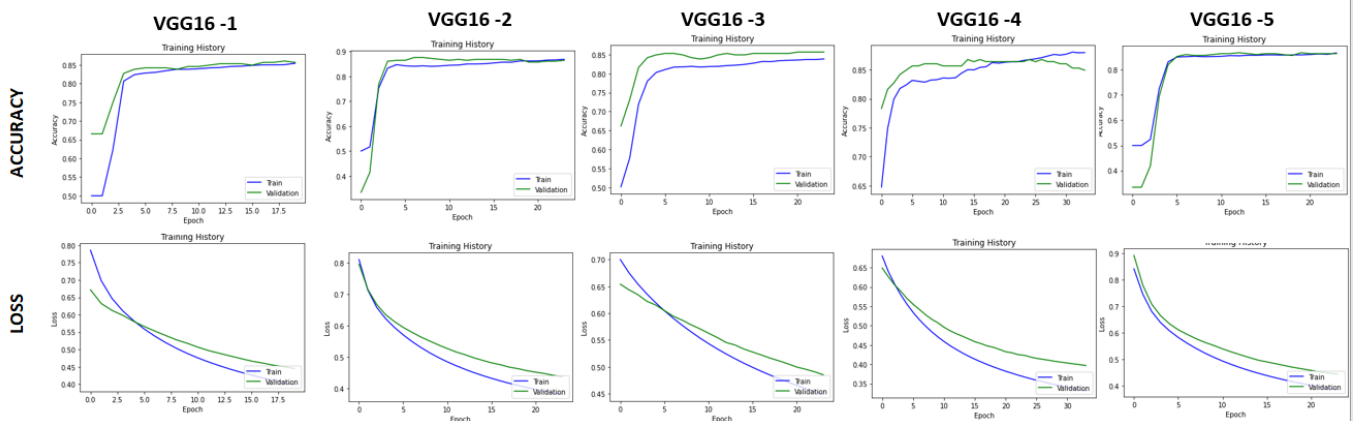
Trainable params: 1,026

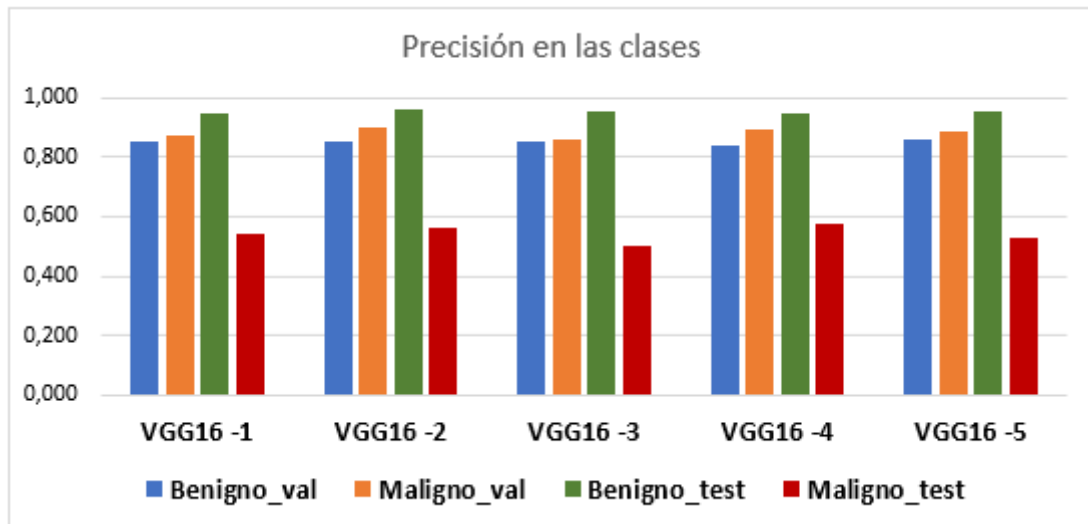
Non-trainable params: 14,714,688

DIAGNÓSTICO DEL RENDIMIENTO. PRUEBAS Y RESULTADOS.

Se han realizado varias iteraciones con una combinación diferente de parámetros, obteniendo los siguientes resultados:

VGG16 - 1	VGG16 - 2	VGG16 - 3	VGG16 - 4	VGG16 - 5
epocas = 20 lr = 0,0002 batch_size = 32 optimizer = Adam	epocas = 24 lr = 0,0002 batch_size = 32 optimizer = Adam	epocas = 24 lr = 0,0001 batch_size = 32 optimizer = Adam	epocas = 34 lr = 0,0002 batch_size = 32 optimizer = Adam	epocas = 24 lr = 0,0002 batch_size = 32 optimizer = Nadam





Las gráficas de accuracy y loss dan buenos resultados. En accuracy, las líneas de train y validación c recen acompasadas a lo largo de las épocas. En el modelo 4, se observa que a partir de la época 2 4 hay una divergencia entre ambas líneas lo que implica que a partir de ese punto el modelo empe ora. Con el optimizador Nadam, el modelo necesita más épocas para conseguir los mismos resulta dos que con Adam. (VGG16-5). Al disminuir considerablemente el parámetro learning_rate del opt imizador Adam se observa que aparece una pequeña brecha entre las líneas de train y validación. Por lo tanto, de los modelos testados, el modelo que ofrece mejores resultados es el VGG16-2.

Todos los modelos no generalizan lo suficientemente bien en la clase “Maligno”. No se supera un 60%. Se va a utilizar una metodología de validación cruzada Group K-Fold para evaluar este rendi miento, por si fuera debido a las particularidades del conjunto de validación seleccionado.

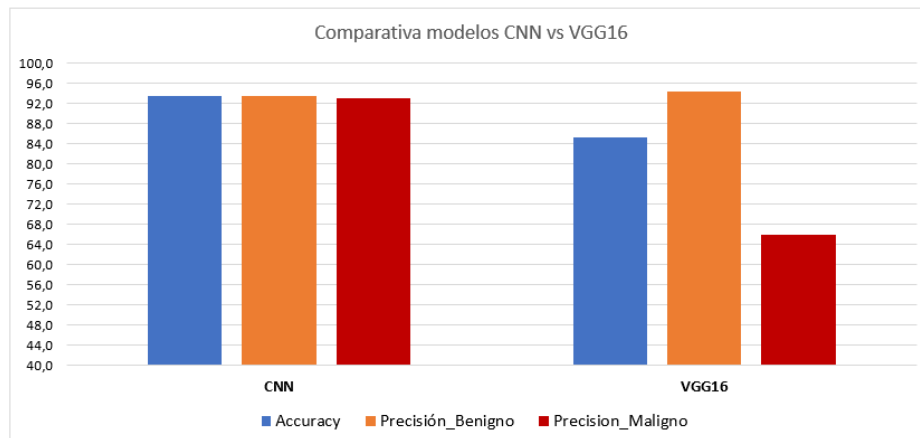
El resultado obtenido ha mejorado el modelo, pero no lo suficiente. El val_accuracy promedio con seguido es de 81.21% durante el entrenamiento. Se ha obtenido un val_accuracy de 85,23% en la primera iteración, 78.63% en la segunda y 79.78%. Al evaluar el modelo sobre el conjunto de test, se ha obtenido un accuracy de 85.26% con una precisión en la clase benigno de 0.9434 y en maligno de 0.66.

Esta metodología no ha conseguido generalizar lo suficientemente bien sobre las imágenes malignas.

SELECCIÓN DEL MODELO.

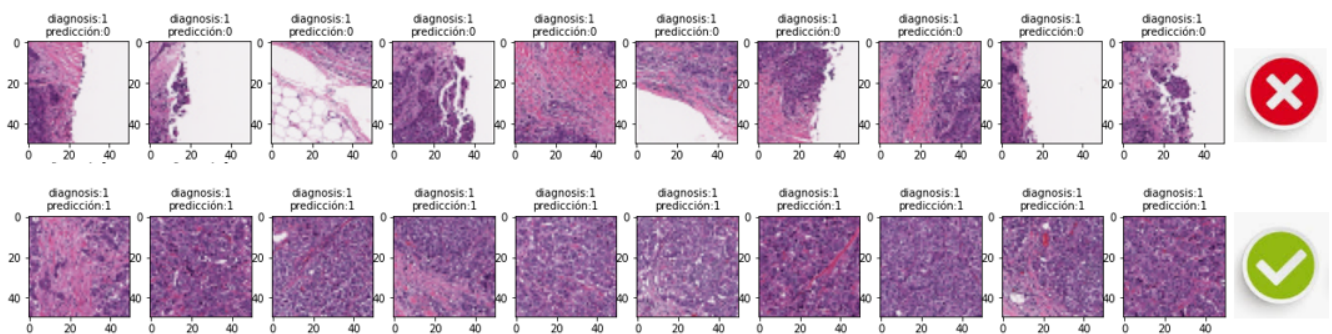
Comparación de resultados.

En la siguiente gráfica se muestran los resultados obtenidos en cada modelo para el conjunto de test.

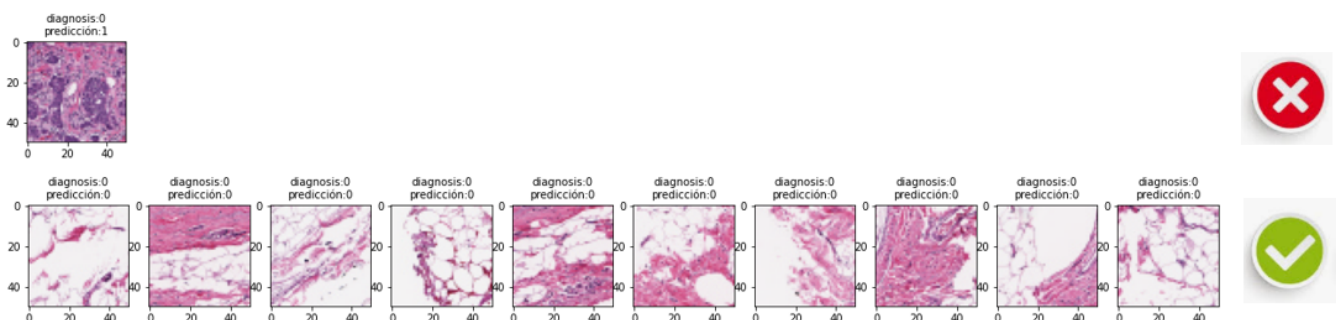


Con el modelo CNN:

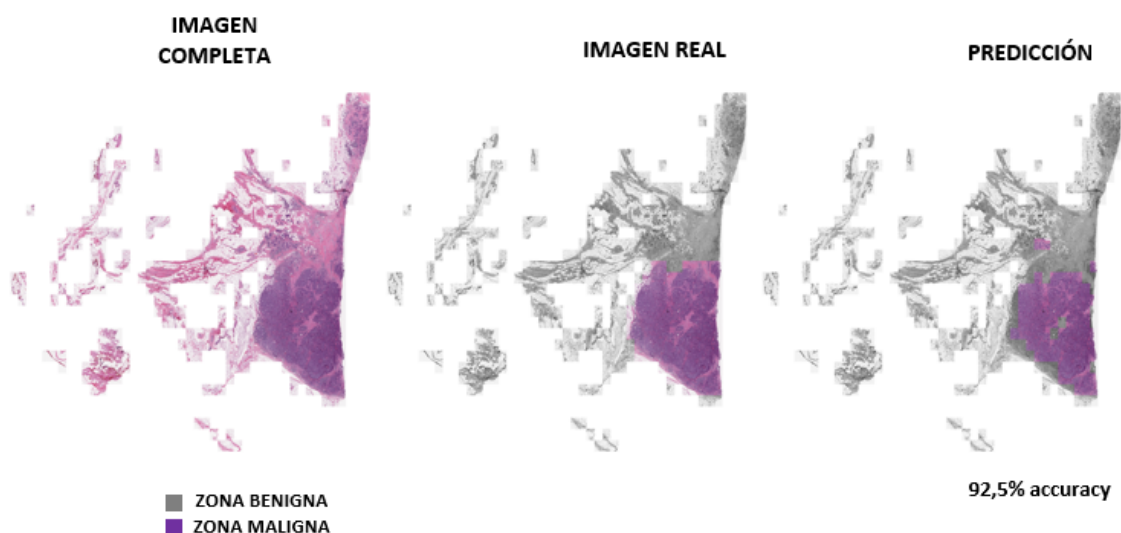
Muestras con diagnóstico **Maligno**, donde el modelo ha fallado y ha acertado.



Muestras con diagnóstico Benigno, donde el modelo ha fallado y ha acertado:

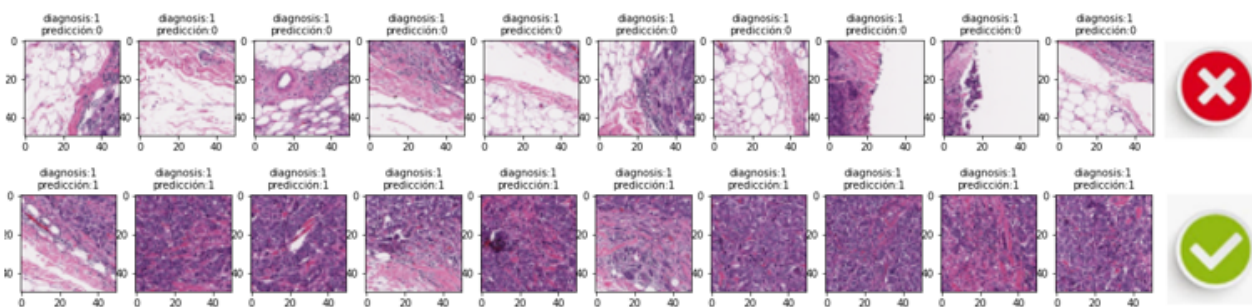


Si vemos la imagen completa de la muestra de tejido mamario del conjunto de test:

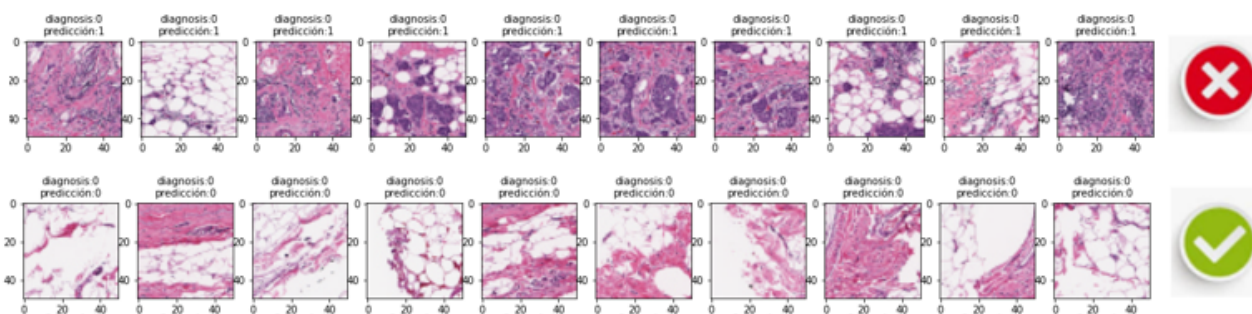


Con el modelo VGG16:

Muestras con diagnóstico **Maligno**, donde el modelo ha fallado y ha acertado.



Muestras con diagnóstico **Benigno**, donde el modelo ha fallado y ha acertado:



Si vemos la imagen completa de la muestra de tejido mamario del conjunto de test:



El modelo CNN ha demostrado tener una mayor capacidad de generalización que el modelo VGG16, prediciendo con más acierto tanto en la clase benigno como en la clase maligno. Por lo tanto, ponemos en producción el modelo CNN.

3.4 Interfaz de usuario

Se ha implementado una red neuronal usando Keras-Tensorflow, que se ejecuta en un servicio web de Flask. Para ello, se ha creado un entorno llamado APIDeep, usando el paquete Anaconda Navigator y en él se han instalado los paquetes necesarios para nuestra aplicación (ver manual de instalación).

Una vez tenemos el servidor corriendo podemos ver la aplicación en cualquier navegador escribiendo en la barra de direcciones, la dirección <http://127.0.0.1:5000>.

Para realizar una predicción o diagnóstico, tenemos que escoger una imagen haciendo clic en el botón “Elegir imagen para diagnóstico”. Cada imagen de nuestro dataset tiene un nombre identificativo, si es de clase 1 significa que es de tipo maligna y si es de tipo 0 entonces benigna.

Una vez cargada la imagen, para realizar la predicción es necesario hacer clic en el botón “Predicción” y después de unos segundos se imprime el resultado en pantalla. Para proceder a obtener nuevas predicciones, debe repetirse el proceso de nuevo.

A continuación se muestra una imagen que contiene cada una de las fases por las que pasa nuestra interfaz.



3.5. Pruebas y mejoras.

Para probar el modelo diseñado se han utilizado las herramientas Colab Collaborative de Google que proporciona recursos para trabajar con GPUs (Unidades de Procesamiento Gráfico) y el programa Jupyter de la plataforma Anaconda que fue creado para desarrollar software de código abierto.

La herramienta Colab ha dado numerosos problemas en cuanto al uso de la RAM. Para solventar este inconveniente se cargaron todas las imágenes en la propia máquina física de cada uno de los miembros del equipo y se ejecutaron varios modelos para comprobar que modelo proporcionaba un mejor accuracy, pero en algunos equipos daba error.

Se han realizado muchas pruebas para ajustar el tamaño de las imágenes y el accuracy obtenido. Al final se optó por escoger el equipo más potente para realizar las pruebas.

4. Proyección del impacto

4.1. Logros y beneficios

Se ha logrado crear el proyecto deseado y mostrarlo a través de una interfaz. Con un val_accuracy promedio de 81.68% durante el entrenamiento, un accuracy de conjunto de test, de 92.52% con una precisión en la clase benigno de 0.935 y en maligno de 0.93.

4.2 Futuras mejoras

Realizar una incorporación de imágenes (carpeta de pacientes) y volver a probar el modelo Vgg16, para mejorar la precisión de imágenes malignas, ya que su accuracy era de 0.66. Y realizar así, las predicciones de las diferentes muestras celulares.

Desarrollar la interfaz de usuario para que sea más útil y pueda incorporar la opción de subir la muestra completa de un paciente, mostrando sobre ella las zonas que el algoritmo de inteligencia artificial ha detectado como cancerígenas.

5. Revisión y comentarios de los miembros del equipo



NAME	REVIEW and COMMENT
AAAA	El proyecto se nos ha hecho bastante intenso, después de muchos fallos con la descompresión de datos, la ram y gpu en colab y tener que empezar de cero, hemos conseguido sacar nuestro proyecto hacia delante.
BBBBB	El proyecto nos ha puesto a prueba constantemente. Ha sido duro e intenso durante prácticamente todo el proceso. Un gran reto, pero finalizado.
CCCCC	Este proyecto ha sido un gran aprendizaje. Ha supuesto todo un reto para nosotras, intenso y duro en ocasiones. Nos llevamos muchas lecciones aprendidas.
DDDDD	Compaginar el curso y el proyecto final con otras obligaciones profesionales y personales ha sido extenuante. Pero finalmente con el apoyo en equipo se ha conseguido salvar obstáculos y alcanzar el objetivo final.